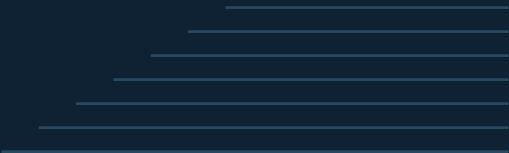


Melting Ice Trackers

40-day production prototype roadmap

October 1 – November 10 | RTX 5050 (8 GB VRAM), 32 GB RAM



Overview

Forty days, from October 1 to November 10, is enough time to build a high-fidelity engineering prototype rather than a rushed hackathon demo. An RTX 5050 with 8 GB of VRAM and 32 GB of RAM can run the full automated processing pipeline end to end.

Data ingestion	Segmentation	Spatial math + forecasting	Dashboard
Days 1-10	Days 11-20	Days 21-30	Days 31-40
Oct 1 – Oct 10	Oct 11 – Oct 20	Oct 21 – Oct 30	Oct 31 – Nov 9

Phase 1: Automated data ingestion Request cloud-free satellite imagery by code through Earth Engine or Planetary Computer.	Phase 2: Segmentation model Train U-Net or DeepLabV3+ with segmentation_models.pytorch and validate on unseen years.
Phase 3: Spatial math + forecasting Turn masks into polygons and true areas, then forecast with time-series models and ERA5 weather.	Phase 4: Dashboard A multi-page monitoring app with a simulated live satellite feed and interactive maps.

Hardware

Component	Specification	Role in the project
GPU	RTX 5050, 8 GB VRAM	Segmentation training and inference in the background pipeline
RAM	32 GB	Holding satellite tiles, masks and time-series data in memory

Phase 1: Automated data ingestion

Days 1-10 (Oct 1 - Oct 10)

Data ingestion Days 1-10	Segmentation Days 11-20	Spatial math + forecasting Days 21-30	Dashboard Days 31-40
Oct 1 - Oct 10	Oct 11 - Oct 20	Oct 21 - Oct 30	Oct 31 - Nov 9

Build an automated data pipeline instead of hardcoding a downloaded Kaggle dataset.

The engine	Google Earth Engine (GEE) Python API or Microsoft Planetary Computer, which let you request cloud-free chunks of satellite imagery programmatically.
The target	Two or three high-profile areas, for example Jakobshavn Isbræ in Greenland, Columbia Glacier in Alaska, or a melting glacier in the Himalayas.
The pipeline step	A script that takes a region's coordinates and a date range, filters out scenes with high cloud cover, extracts the Sentinel-2 or Landsat 8 bands, and saves them locally.



Phase 2: High-fidelity segmentation model

Days 11-20 (Oct 11 - Oct 20)

Data ingestion Days 1-10	Segmentation Days 11-20	Spatial math + forecasting Days 21-30	Dashboard Days 31-40
Oct 1 - Oct 10	Oct 11 - Oct 20	Oct 21 - Oct 30	Oct 31 - Nov 9

With ten days dedicated to AI, the team can train a robust model instead of taking shortcuts.

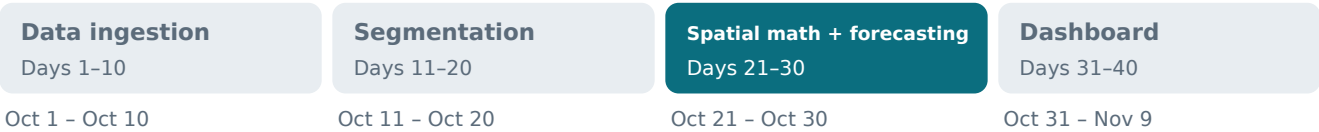
Architecture	U-Net or DeepLabV3+ using segmentation_models.pytorch, with a heavy backbone such as ResNet-50 or EfficientNet-B4.
Hardware execution	Runs on the RTX 5050. Days 11-14 go to tuning hyperparameters: learning rates, augmentations such as random flips and rotations with Albumentations, and a combined Dice + BCE loss to handle edge boundaries smoothly.
Validation	Hold out a completely separate set of years, for example train on 2018-2024 and test on 2025-2026, to prove the model holds up over time.

Temporal validation split

2018	2019	2020	2021	2022	2023	2024	2025	2026
Train							Test	

Phase 3: Spatial math and advanced forecasting

Days 21–30 (Oct 21 – Oct 30)

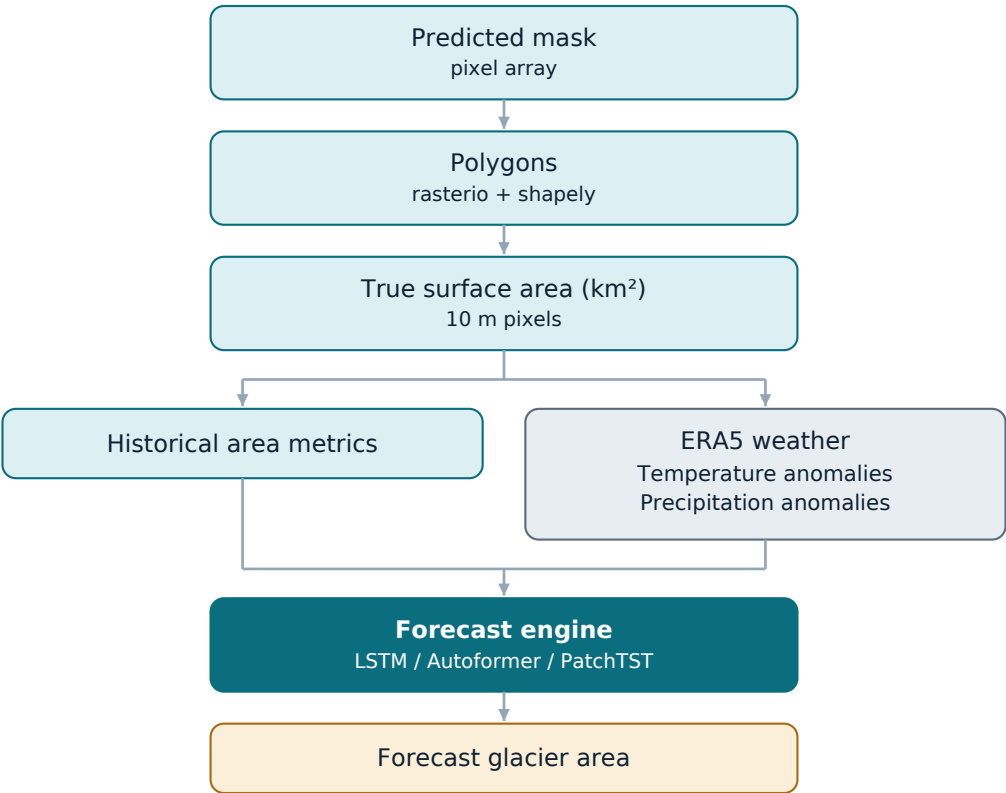


The geo-math

Use rasterio and shapely to convert each predicted mask from a pixel array into real geospatial polygons, then calculate the true surface area from the satellite's spatial resolution (10 m per pixel for Sentinel-2).

The forecast engine

Instead of basic linear tracking, use an LSTM network or an Autoformer / PatchTST time-series transformer that processes historical area metrics together with weather factors such as temperature and precipitation anomalies from Copernicus ERA5.



Phase 4: Full-scale dashboard architecture

Days 31-40 (Oct 31 - Nov 9)

Data ingestion Days 1-10	Segmentation Days 11-20	Spatial math + forecasting Days 21-30	Dashboard Days 31-40
Oct 1 - Oct 10	Oct 11 - Oct 20	Oct 21 - Oct 30	Oct 31 - Nov 9

Build an interface that simulates an active monitoring platform.

The UI	A multi-page Streamlit app, or a FastAPI + React application.
“Real-time” simulation	Satellites revisit every few days, so simulate a live system. When a new raw .tif or .png image is dropped into incoming_satellite_feed/, the dashboard picks it up, runs it through the GPU pipeline in the background, updates the historical CSV and redraws the graph.
Interactive maps	streamlit-folium or Mapbox, letting users zoom into the glacier and toggle the AI-detected ice boundary on and off.



Key Python libraries

Install these in the project's virtual environment.

```
# Data & geospatial processing
pip install earthengine-api rasterio shapely geopandas numpy pandas

# Deep learning & modeling
pip install torch torchvision albumentations segmentation-models-pytorch statsmodels

# Interface & visualization
pip install streamlit streamlit-folium plotly matplotlib
```

Group	Libraries	Used in
Data & geospatial	earthengine-api, rasterio, shapely, geopandas, numpy, pandas	Phases 1 and 3
Deep learning & modeling	torch, torchvision, albumentations, segmentation-models-pytorch, statsmodels	Phases 2 and 3
Interface & visualization	streamlit, streamlit-folium, plotly, matplotlib	Phase 4

Immediate next steps

Days 1–3: don't touch deep learning models yet. First secure access to the data.

- 1

Register for Google Earth Engine
Sign up for a developer account. Approval can take from a few minutes to a couple of days, so do this first.
- 2

Verify the geospatial setup
Write a simple Python script to confirm rasterio and geopandas install and import cleanly without environment conflicts.

Next milestone: a Python script that connects to Google Earth Engine and pulls down cloud-filtered imagery for the chosen glacier coordinates.